

Source → filter() → map() → (optional sorted/distinct) → Terminal Operation		
Terminal Operation	Use Case	One-Line Code Example
collect()	Accumulate elements	<code>list.stream().collect(Collectors.toList());</code>
collect()	Transform & group	<code>users.stream().collect(Collectors.groupingBy(User::getDept));</code>
collect()	Map → Reduce → Store	<code>orders.stream().collect(Collectors.groupingBy(Order::getType, Collectors.summingDouble(Order::getAmount)));</code>
-		
forEach()	Logging	<code>list.stream().forEach(x -> logger.info(x.toString()));</code>
forEach()	Printing	<code>list.stream().forEach(System.out::println);</code>
forEach()	Updating external systems	<code>users.stream().forEach(u -> auditService.update(u));</code>
reduce()	Combine values	<code>list.stream().reduce(0, Integer::sum);</code>
reduce()	Aggregate (sum / max / concat)	<code>list.stream().reduce(Integer::max).orElse(0);</code>

-		
count()	No custom logic allowed	list.stream().count();
-		
findFirst()	Conditional check before first match	list.stream().filter(x -> x > 10).findFirst();
-		
allMatch()	Validation logic	users.stream().allMatch(u -> u.getAge() >= 18);
allMatch()	Rule checks	orders.stream().allMatch(o -> o.isPaid());
-		
anyMatch()	Existence checks	users.stream().anyMatch(u -> u.getRole().equals("ADMIN"));
noneMatch()	Negative validation	users.stream().noneMatch(u -> u.isBlocked());